

Advanced Natural Language Processing: AdvNLP

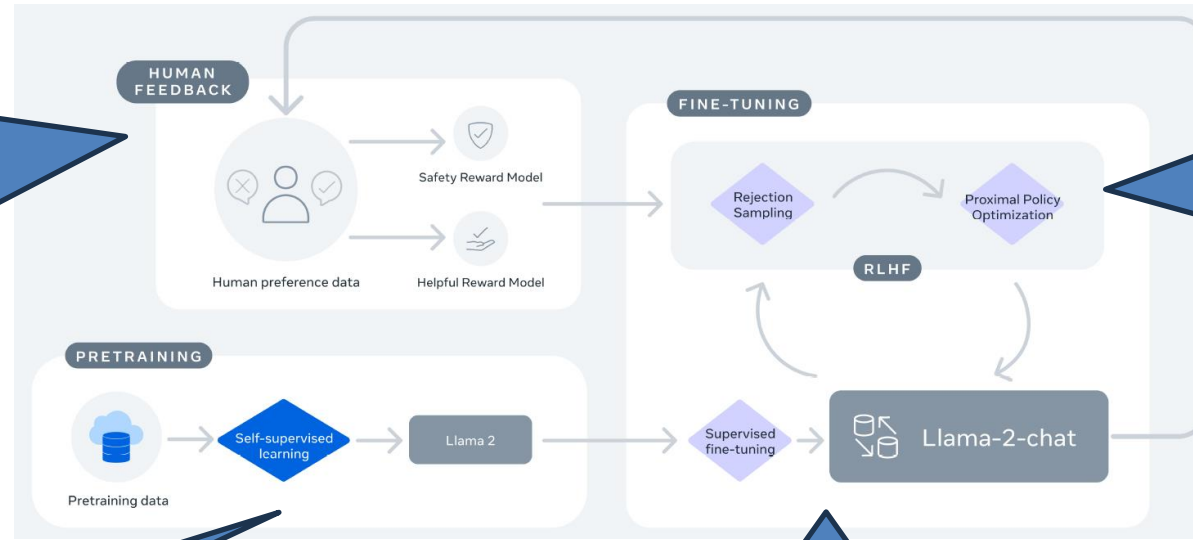
*Lesson 9: LLM in-context learning &
prompt engineering*

Table of Contents

1. Introduction
2. In-context learning (ICL)
3. Prompt engineering: structure and roles
4. Extensive answers with chain-of-thought (CoT) prompting
5. Using LLMs: model servers

1. INTRODUCTION

Reminder: how to train a LLM to follow instructions helpfully and safely



(3) Auxiliary model learns to predict if a response is *helpful* or *safe* on datasets with human ratings ($3 \cdot 10^6$, 50% public)

(1) Learn to predict the next word on a dataset of about 10^{12} words

(2) Same as (1), but on pairs of (*instruction*, *response*) written by human ($\sim 10^4$ to 10^6)

(4) Fine-tune the LM so that it improves its helpfulness and safety scores (using RS et PPO, 5 iterations)

Principle of RS : same as (1), but on (*instruction*, *response*) pairs with high *helpfulness/safety*

From text completion to intelligent behavior

- Language models were initially developed for next token prediction
- How do we get from next token prediction to problem solving?
- Can we expect an LLM to directly write a solution to any problem?

The sky is ...
LLM → blue

The odd numbers in this list
add up to an even number:
15, 32, 5, 13, 82, 7, 1.

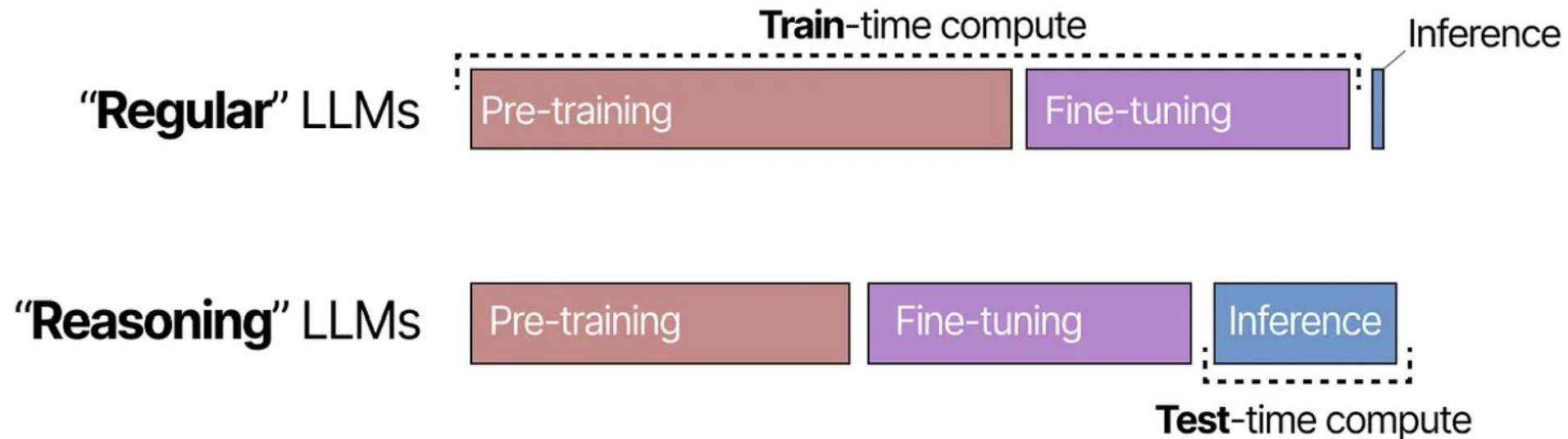
Prompt

LLM → Adding all the odd
numbers (15, 5, 13, 7, 1) gives 41.
The answer is false.

Answer

The way to build a nuclear
fusion reactor is:
LLM → ?

Can we improve capabilities of a LLM after it was trained and fine-tuned? Yes, we can!



Quality of answers is improved if more compute is used when generating them

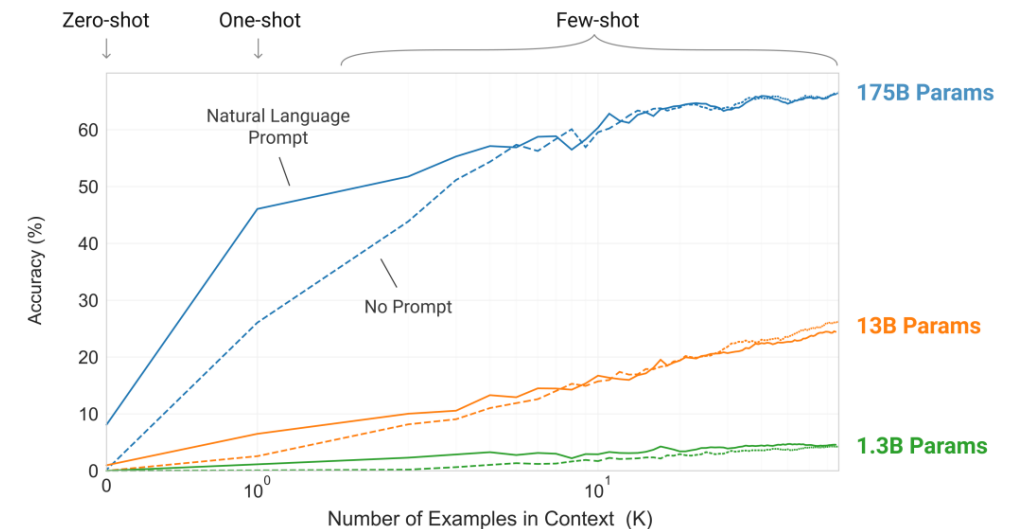
1. Increase the "input size" → long prompts that describe the problem better: ICL
2. Increase the "output size" → instruct model to generate longer answers: CoT

2. IN-CONTEXT LEARNING (ICL)

In-context learning as an emergent capability of LLMs

- In-context learning (ICL): conditioning a GPT-3 model with 175B parameters on **instructions (in natural language) and examples** of a task showed high LLM improvements across diverse tasks
- Performance improved for larger models and when presenting few examples (few-shot learning)

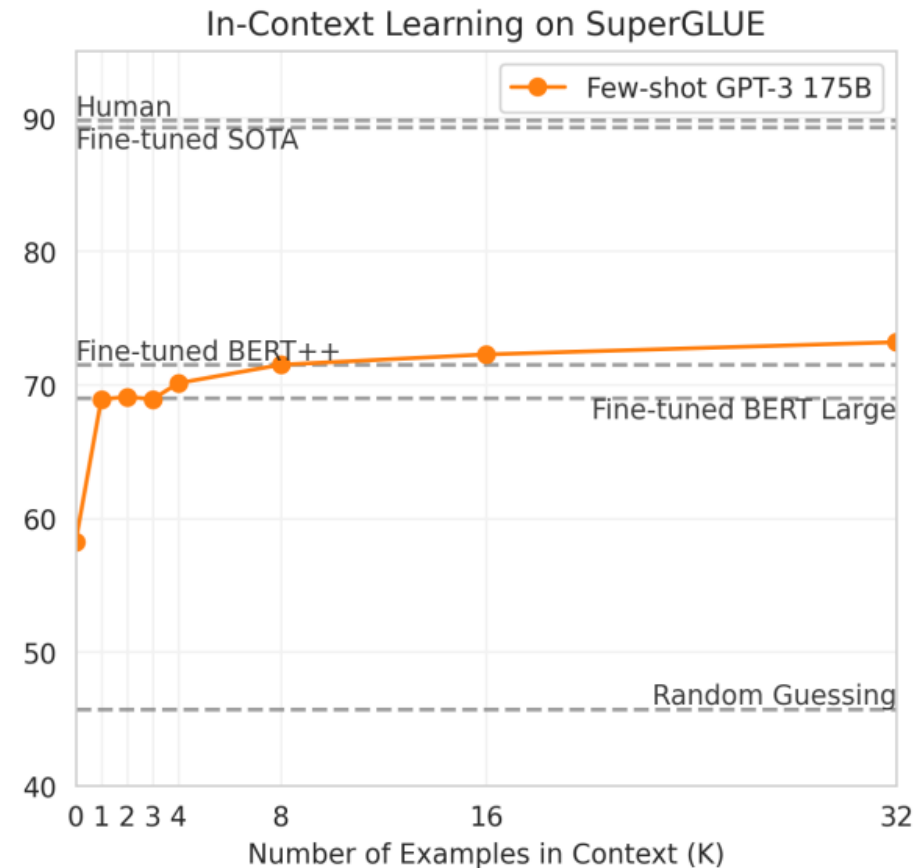
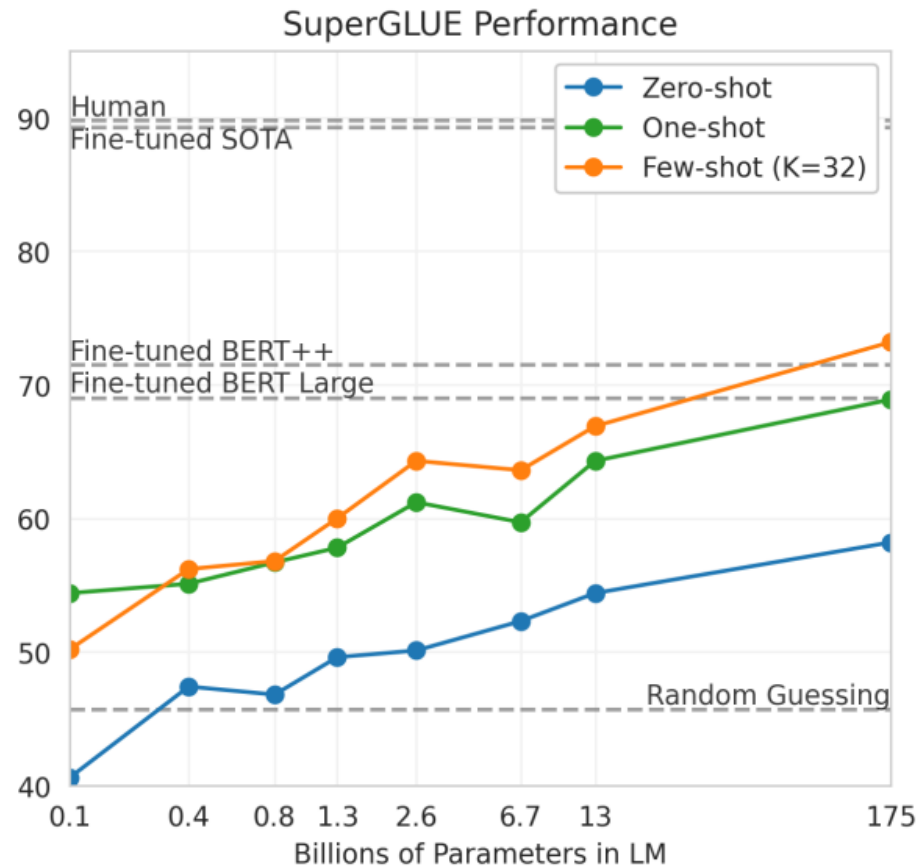
Example: GPT-3 asked to remove symbols from a word



Source: [Brown T. et al. \(2020\), Language Models are Few-Shot Learners](#)

In-context learning as an emergent capability of LLMs

GPT-3 asked to resolve eight different SuperGLUE language understanding tasks



Source: [Brown T. et al. \(2020\), Language Models are Few-Shot Learners](#)

In-context learning – using examples in prompts

- LLMs can perform many tasks with no need for examples: ‘zero-shot’ prompting
 - PROMPT = Classify the text into neutral, negative or positive.
Text: I think the vacation is okay.
Sentiment:
 - OUTPUT = Neutral
- Giving examples is a good strategy to improve performance on more complex tasks
 - called ‘one-shot’ or ‘few-shot’ prompting
 - use 2—5 informative, yet concise examples

PROMPT	<instructions>
	"" <data1> ""
	<desired response to 1>
	"" <data2> ""
	<desired response to 2>
	"" <data3> ""



Response	<response to 3>
----------	-----------------

Best practice of few-shot prompting

- Selection of **informative examples** is key, selection can be manual or even with unsupervised or supervised learning
- **Drawbacks** of few-shot prompting
 - Model output can be **biased** toward the examples that were presented
 - Few-shotting can **induce spurious patterns** through extrapolation from a few examples
 - **Scaling** can be an issue if task requires **a large context**

1. Exemplar Quantity Include as many exemplars as possible* ✓ Trees are beautiful: Happy I hate Pizza: Angry Squirrels are so cute: Happy YouTube Ads Suck: Angry I'm so excited:	2. Exemplar Ordering Randomly order exemplars* I am so mad: Angry I love life: Happy I hate my boss: Angry Life is good: Happy I'm so excited:
3. Exemplar Label Distribution Provide a balanced label distribution* ✓ I am so mad: Angry I love life: Happy I hate my boss: Angry Life is good: Happy I'm so excited:	4. Exemplar Label Quality Ensure exemplars are labeled correctly* I am so mad: Angry I love life: Happy I hate my boss: Angry Life is good: Happy I'm so excited:
5. Exemplar Format Choose a common format* ✓ Im hyped!: Happy Im not very excited: Angry I'm so excited:	6. Exemplars Similarity Select similar exemplars to the test instance* Im hyped!: Happy Im not very excited: Angry I'm so excited:

Why does ICL work? [optional slide]

There is no **single agreed-upon mechanism** to explain ICL but several lines of research

Pre-training stage

- Data distribution, domain and diversity strongly shape ICL ability and generalization across tasks

Inference stage

- Prompt design choices and demonstration presentation, i.e., selection, order, label-space coverage and format of examples strongly impact ICL ability

Learning mechanisms

- Specific attention heads (aka **induction heads**) can replicate previous patterns for next token prediction; discrete demonstration labels (e.g., “Review: ... Sentiment: Positive”) serve as anchors, which aggregate and distribute key information
- ICL is based on implicit **Bayesian inference**, i.e., a shared latent concept among examples is identified
- ICL is based on **meta-gradients** constructed from input / label patterns and applied via attention **activation modifications** in the forward pass (without weight modification)

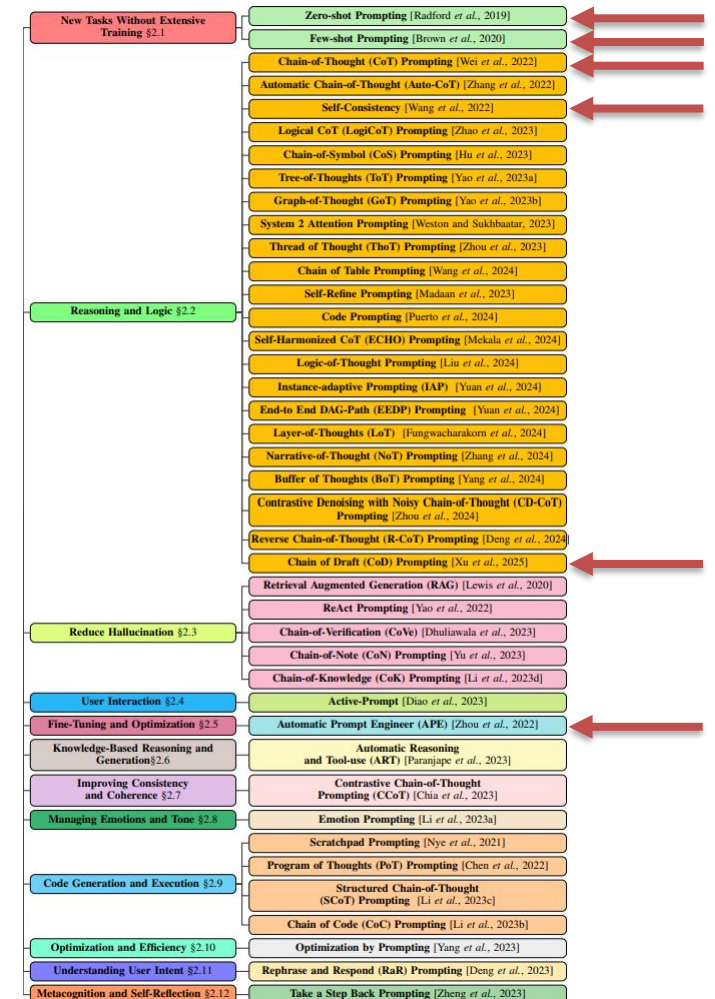
Source: [Dong X. et al. \(2023\), A Survey on In-context Learning](#)

3. PROMPT ENGINEERING: STRUCTURE AND ROLES

Prompt engineering – definition

- Prompt engineering is the process of **structuring natural language input instructions** or prompts to produce specified outputs from a generative AI model, e.g., an LLM
- Designing and refining prompts in order to produce more accurate, relevant, or useful outputs involves **understanding how a model interprets language**
- Effective prompt engineering also requires **setting LLM configuration parameters optimally** for the task given (cf. decoding methods)

Taxonomy of Prompting Techniques



Anatomy of LLM prompts – three main roles

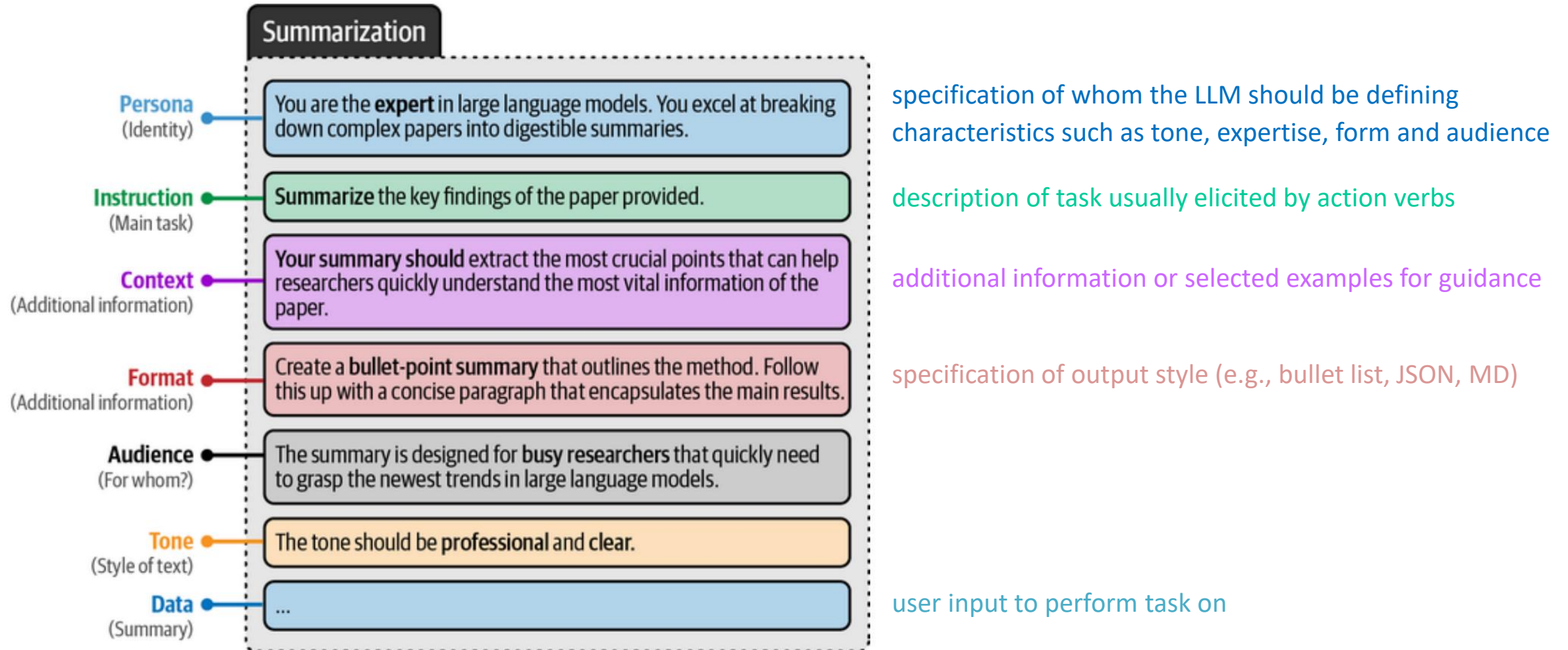
Prompts can (and should) be structured in several parts, called ‘roles’

1. ‘Developer’: also called ‘System’
 - high-level instructions that guide the behavior of the LLM
 - define the persona or the personality traits (tone, expertise, form)
2. ‘User’: main section with instructions
 - a. definition of the task: e.g., request, query, set of instructions
 - b. data on which to perform the task (if needed)
 - c. additional specifications **as seen in the next slide**
3. ‘Assistant’: optional
 - the text that the model has actually generated in a previous round, or, alternatively, what the model should generate in response to an instruction (**few-shot prompting**)

Typical use: System + [User + Assistant]* + User → LLM → Assistant’s response

Zero or more preceding user messages
and LLM answers (actual or desired)

Anatomy of LLM prompts – important elements

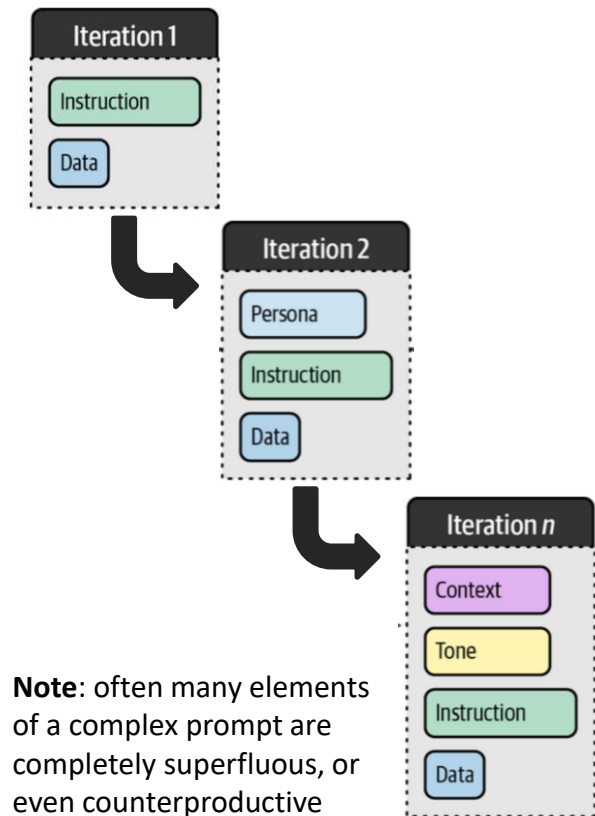


Prompt engineering – best practices

- Build a prompt step-by-step through experimentation, and complexify it by adding elements, but avoid overcrowding it
- **Design with simplicity**: make prompts concise, use verbs to describe actions
- **Provide examples**: showcase desired outputs, so that the model generates accordingly
- **Be specific about the output**: add details about desired outputs, specify steps to complete a given task, instruct model not to fabricate output if answer is unknown
- **Use instructions over constraints**: positive instructions can be more effective than relying heavily on negative constraints (constraints against harmful or biased content are OK)
- **Control the response length**: configure the max token limit parameter or explicitly request a specific output length (e.g., “summarize in one sentence”)
- **Experiment with formats**: **input** – vary the structure prompts, use different formats; **output** – try JSON, MD or XML for non-creative tasks

Prompt engineering – construction & testing

- Prompt engineering is an **iterative constructive process** and involves repeated testing to identify issues, e.g.,
 - verify labeling of given texts via ground truth, AI model or rules
 - quantify rate of successful logical reasoning or math solution
 - flag scenarios where unsafe or undesirable results are returned
 - assess robustness against known prompt injection
- Engineering and testing prompts can lead to radical decreases in cost and latency of your production system



Prompt engineering – programmatic testing

- Use prompt templates
 - build prompts automatically: static text + placeholders
 - substitute data into placeholders at runtime
 - create many instances of a task, with different data
 - the [LangChain](#) library helps building prompt templates
- Perform systematic tests
 - representative of real-world usage
 - large sample sizes for statistical power
 - easy to automate
 - use rating by humans or by LLMs 😊
 - LLMs can estimate if an output is close to a gold-standard

```
import { ChatPromptTemplate } from "@langchain/core/prompts";

const promptTemplate = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful assistant"],
  ["user", "Tell me a joke about {topic}"],
]);

await promptTemplate.invoke({ topic: "cats" });
```

Source: LangChain, [Prompt templates](#)

DIFFERENCE TO DETECT	SAMPLE SIZE NEEDED FOR 95% CONFIDENCE
30%	~10
10%	~100
3%	~1,000
1%	~10,000

Source: OpenAI, [Prompt engineering](#)

Documentation of prompts

Name	[name and version of your prompt] PP-JSON-09		
Goal	[One sentence explanation of the goal of this attempt] e.g., Parse pizza orders to JSON		
Model	[name and version of the used model] e.g., Qwen3-4B-Thinking-2507-FP8		
Temperature	[value between 0 - 1] e.g., 0.1	Token Limit	[number] e.g., 250
Top-K	[number] e.g., NA	Top-P	[number] e.g., 1
Prompt	[Write all the full prompt]		
Output	[Write out the output or multiple outputs]		

Parse a customer's pizza order into valid JSON:

EXAMPLE: I want a small pizza with cheese, tomato sauce, and ham.

JSON Response: `` `{"size": "small", "type": "normal", "ingredients":
[["cheese", "tomato sauce", "ham"]] } ```

Now, I would like a large pizza, with the first half cheese and mozzarella. And the other tomato sauce, peperoni and pineapple.

JSON Response:

`` ` { "size": "large", "type": "half-half", "ingredients":
[["cheese", "mozzarella"], ["tomato sauce", "peperoni", "pineapple"]] } ```

Use of roles: Zero-shot vs. one-shot

System	Answer in a consistent style.	Answer in a consistent style.
User	Teach me about the ocean.	Teach me about patience.
Assistant		The river that carves the deepest valley flows from a modest spring; the grandest symphony originates from a single note; the most intricate tapestry begins with a solitary thread.
User		Teach me about the ocean.
Assistant		
User		
RESPONSE	The ocean is a vast body of saltwater that covers more than 70% of Earth's surface. It plays a crucial role in regulating the planet's climate, absorbing carbon dioxide, producing oxygen, and supporting a wide variety of marine life.	The ocean, vast and immeasurable, is a symbol of both power and tranquility. It holds within its depths mysteries untold and creatures unseen, representing both the known and the unknown in perfect harmony. Just as the ocean ebbs and flows with the tides, so too does life present us with challenges and opportunities, reminding us to adapt and embrace change.

4. EXTENSIVE ANSWERS WITH CHAIN-OF-THOUGHT (CoT) PROMPTING

Few-shot vs. chain-of-thought

System	You are a helpful assistant.	You are a helpful assistant.
User	The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.	The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.
Assistant	Adding all the odd numbers (11, 13) gives 24. The answer is True.	Let's think step by step.
User	The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.	
Assistant	Adding all the odd numbers (17, 9, 13) gives 39. The answer is False.	
User	The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.	
RESPONSE	Adding all the odd numbers (15, 5, 13, 7, 1) gives 41. The answer is False.	1. Identify the odd numbers in the group: 15, 5, 13, 7, 1 2. Add up the odd numbers: $15 + 5 + 13 + 7 + 1 = 41$ 3. Check if the sum is an even number: 41 is an odd number, so there seems to be a mistake in the statement that the odd numbers add up to an even number.

Chain-of-Thought prompting

One-shot prompt

Prompting with a single example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The answer is 27. ❌

Chain-of-thought prompt

Prompting with a reasoning example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.
 $5 + 6 = 11$
The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✅

Example

Reasoning process (thought)

Instruction

Reasoning process (thought)

Zero-shot chain-of-thought

Prompting without example

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Let's think step-by-step.

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✅

Instruction

Prime reasoning

Reasoning process (thought)

No example provided by user!

LLMs memorize training data and match patterns: this looks like **thinking** or **reasoning**

Chain-of-Thought prompting

- Instruct the model to write a longer, more explicit answer: enforcing coherence between detailed sentences leads to a correct solution
 - this is seen as a form of “reasoning” or “thinking”
- Zero-shot CoT = add “Let’s think step by step”
- Step-by-step CoT = ask the model to generate intermediary results for several steps
- ❖ Some models have been trained to generate CoT answers and finally to summarize them
 - the CoT answer can be hidden from the user if tagged by the model (playing the role of an “inner monologue”): e.g., GPT4-o1 or DeepSeek-R1

Example: step-by-step CoT for tutoring

SYSTEM	Follow these steps to answer the user queries. Step 1 - First work out your own solution to the problem. Don't rely on the student's solution since it may be incorrect. Enclose all your work for this step within triple quotes (""). Step 2 - Compare your solution to the student's solution and evaluate if the student's solution is correct or not. Enclose all your work for this step within triple quotes (""). Step 3 - If the student made a mistake, determine what hint you could give the student without giving away the answer. Enclose all your work for this step within triple quotes (""). Step 4 - If the student made a mistake, provide the hint from the previous step to the student (outside of triple quotes). Instead of writing "Step 4 - ..." write "Hint:".
USER	Problem Statement: <insert problem statement> Student Solution: <insert student solution>

Source: OpenAI, [Prompt engineering](#)

Chain-of-Draft prompting

- Chain-of-Draft (CoD) prompts generate minimalistic, informative intermediate reasoning outputs
 - very short ($\approx \leq 5$ words)
 - contains only critical information (numbers, relations, keywords)
- CoD matches or surpasses CoT in accuracy, reduces latency and cost (CoD uses much less tokens than CoT i.e. $< 10\%$)

Chain-of-Thought

Think step by step to answer the following question. Return the answer at the end of the response after a separator ####.

Chain-of-Thought

Q: Jason had 20 lollipops. He gave Denny some lollipops. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?
 A: Let's think through this step by step:
 1. Initially, Jason had 20 lollipops.
 2. After giving some to Denny, Jason now has 12 lollipops.
 3. To find out how many lollipops Jason gave to Denny, we need to calculate the difference between the initial number of lollipops and the remaining number.
 4. We can set up a simple subtraction problem: Initial number of lollipops - Remaining number of lollipops = Lollipops given to Denny
 5. Putting in the numbers: $20 - 12 = \text{Lollipops given to Denny}$
 6. Solving the subtraction: $20 - 12 = 8$
 Therefore, Jason gave 8 lollipops to Denny.
 ####
 8 lollipops

Chain-of-Draft

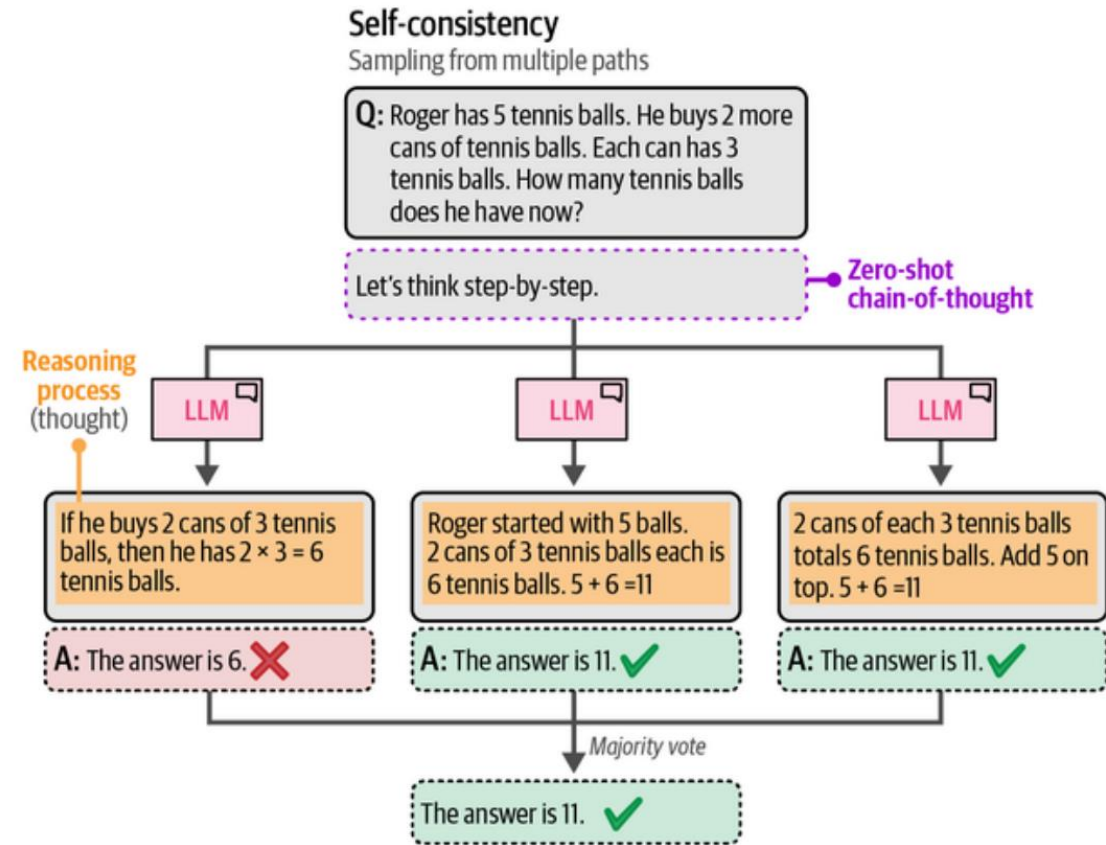
Think step by step, but only keep a minimum draft for each thinking step, with 5 words at most. Return the answer at the end of the response after a separator ####.

Chain-of-Draft

Q: Jason had 20 lollipops. He gave Denny some lollipops. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?
 A: $20 - x = 12$; $x = 20 - 12 = 8$. #### 8

Self-consistency prompting

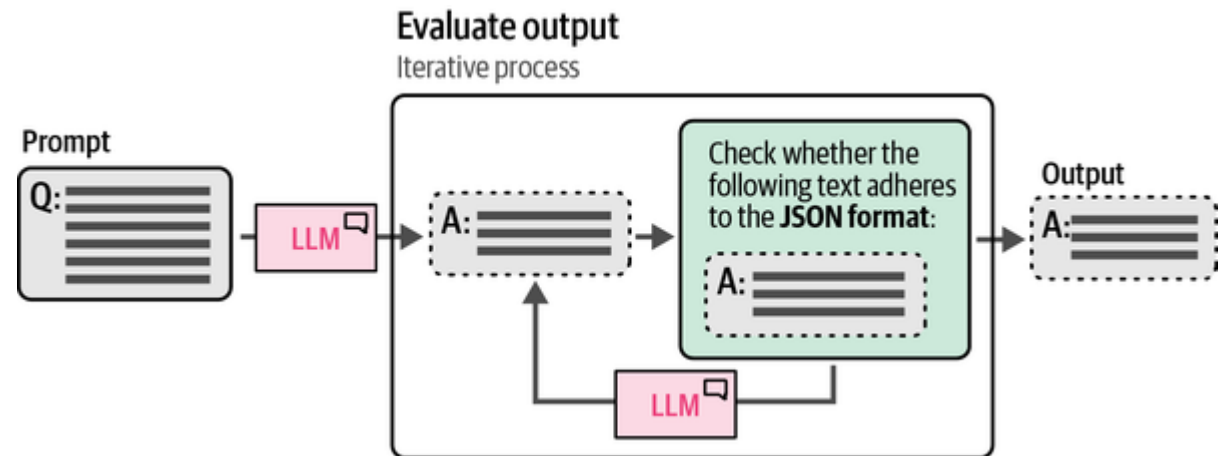
- LLM decoding strategies may introduce randomness and hence reduce response quality
- Mitigation option
 - use one prompt
 - call LLM repeatedly with changing parameters, e.g., temperature or top-p
 - use majority voting to build final response
- This involves multiple LLM calls



Source: J. Alammari & M. Grootendorst, [Hands-On Large Language Models](#), O'Reilly, 2024

Constrained sampling prompting

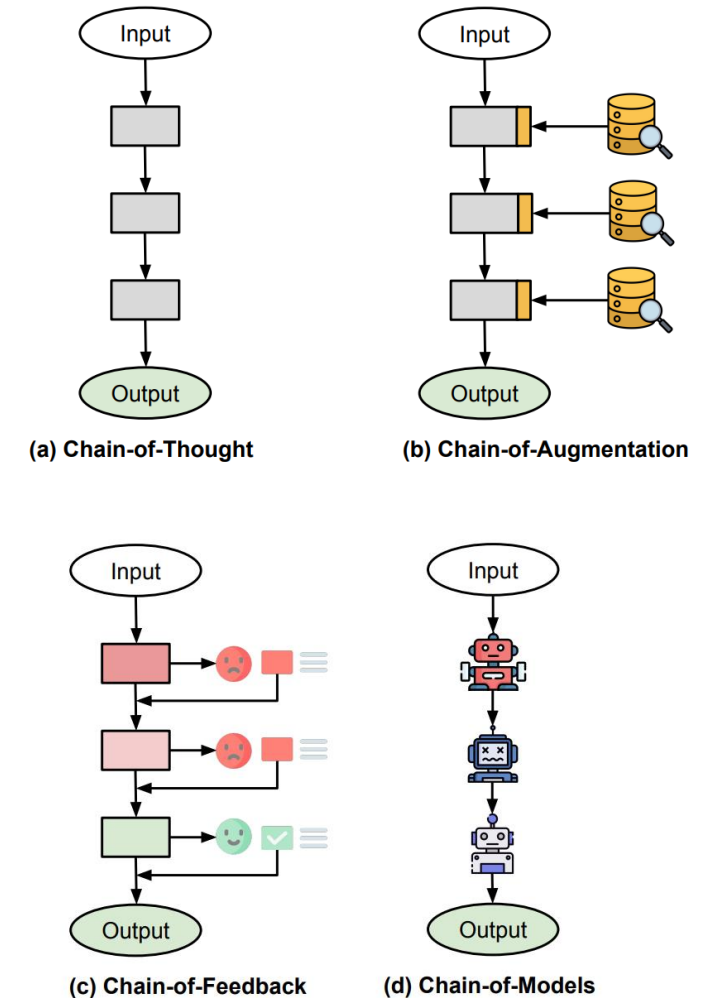
- LLM might not follow instructions precisely and hence reduce response quality
- Mitigation option
 - Specify relevant guardrails
 - Call LLM and evaluate output against predefined guardrails
- This involves another LLM call
- NB: Rules can also be specified before to constrain the token selection process



Source: J. Alammar & M. Grootendorst, [Hands-On Large Language Models](#), O'Reilly, 2024

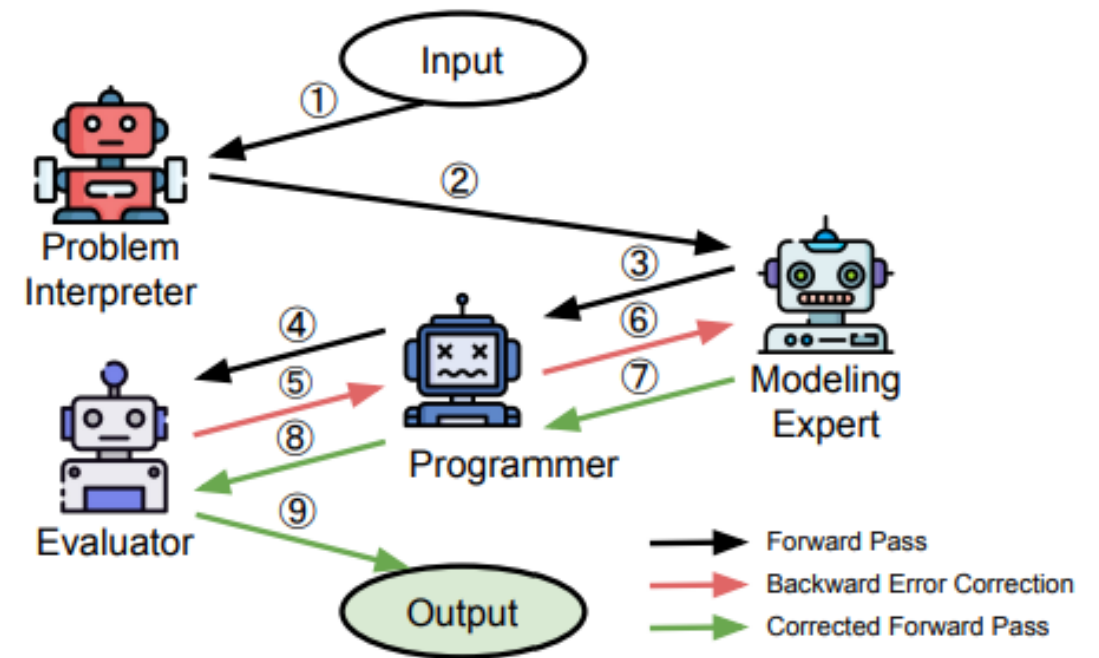
Chain-of-*X* prompting

- Chaining has emerged as general paradigm to break up problems into n steps
 - Chain-of-Augmentation**: iteratively uses outputs of previous steps to generate instructions for the next step, e.g.
 - ReAct** reasons and retrieves external knowledge or
 - Chain-of-Opinion** analyzes historical user opinions to predict future user reactions
 - Chain-of-Feedback**: uses feedback interlaced with the generation process to improve and refine responses, e.g.
 - Chain-of-Hindsight** to improve human preference alignment or
 - Chain-of-Verification** plans several verification questions based on the initial responses and then answer verification questions)



Chain-of-X prompting

- Newer approaches go beyond using one single model
 - **Chain-of-Models**: constructs a chain of models and leverages distinct strengths of each model, e.g.
 - **Chain-of-Experts** involves a consortium of expert LLMs working in sequence, each contributing its specialized knowledge to build upon the results developed
 - **Chain-of-Discussion** employs multiple LLMs engaging in a dialogue, critiquing and refining each other's contributions before reaching a consensus in the final response



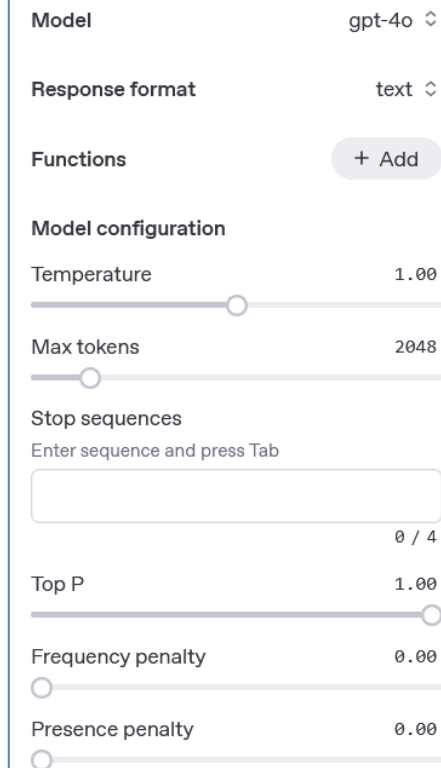
Source: Y. Xia et al., 2025 [A Survey of Chain-of-X Paradigms for LLMs](#)

5. USING LLMs: MODEL SERVERS

How do we input/output text to/from an LLM?

- Input methods
 - chat interface (e.g., ChatGPT or DeepSeek)
 - function calls to a local model (e.g. with the ‘transformer’ library from Hugging Face)
 - calls to the API of a proprietary model (e.g., GPT-4o)
 - calls to a model server (e.g., Llama.cpp or Ollama)
- Setting the parameters of the decoder
 - temperature t ; p in top-p or k in top-k
 - maximum sequence length
 - stop sequence of tokens (if it is generated, then stop)

Example: GPT playground



The screenshot shows the GPT playground interface with the following settings:

- Model:** gpt-4o
- Response format:** text
- Functions:** + Add
- Model configuration:**
 - Temperature:** 1.00
 - Max tokens:** 2048
 - Stop sequences:** Enter sequence and press Tab (0 / 4)
 - Top P:** 1.00
 - Frequency penalty:** 0.00
 - Presence penalty:** 0.00

How do we provide prompt structure to an LLM?

1. Implicitly (early models)

- roles are distinguished using ad-hoc formatting: names + separators
 - the LLM understands them because it has seen them in training data

```
###  
User: ...  
###  
Assistant: ...
```

2. Explicitly (most models)

- roles are distinguished using tags, e.g. `<|user|>` or `<|assistant|>`, documented for each LLM
 - the LLM has learned the meaning of the tags during instruction-tuning
- **Encoding the structure of a prompt** – this depends on how the LLM is used
 - chat interface: **implicit** text formatting, possibly with explicit tags
 - graphical interface: distinct input fields for each role
 - API: **prompt structured as JSON** (e.g., OpenAI's ChatML)
 - note: the GUI or API convert the JSON structure to explicit tags, because the input to the LLM is always a string

Examples of prompts to GPT-4o via Python API

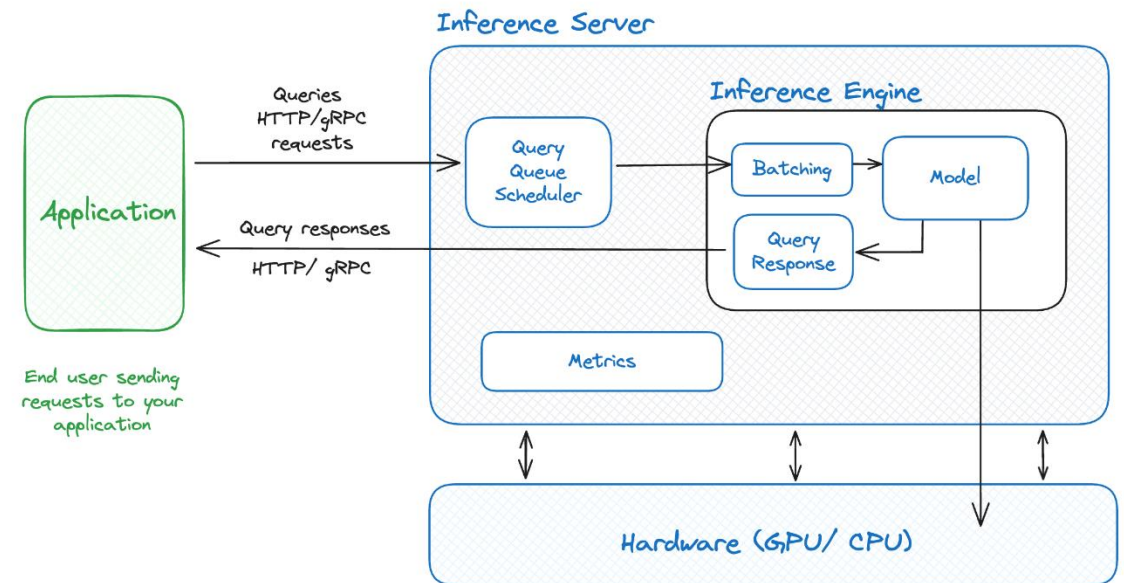
```
1 from openai import OpenAI
2 client = OpenAI()
3
4 response = client.chat.completions.create(
5     model="gpt-4o",
6     messages=[
7         {
8             "role": "system",
9             "content": "You will be provided with statements, and your task is to convert them to standard English."
10        },
11        {
12            "role": "user",
13            "content": "She no went to the market."
14        }
15    ],
16    temperature=1,
17    max_tokens=256,
18    top_p=1
19 )
```

```
1 from openai import OpenAI
2 client = OpenAI()
3
4 completion = client.chat.completions.create(
5     model="gpt-4o",
6     messages=[
7         {"role": "developer", "content": "You are a helpful assistant."},
8         {
9             "role": "user",
10            "content": "Write a haiku about recursion in programming."
11        }
12     ]
13 )
```

Source: <https://platform.openai.com/docs/examples>

Model servers

- LLMs can be called with functions (e.g., Python) or, more often, run as a **server**
- Advantages
 - servers can use multiple GPUs
 - multiple clients can use the same server → optimized memory usage
 - a client application can use various LLMs on various servers
- Servers are often made of two parts
 - **inference engine** = run LLMs, generate text
 - **inference server** = handle requests, collect metrics
 - metrics: throughput (req/sec), latency (sec)



LLM solutions: client-server architectures

- **Local Servers (Open-Source Software)** allowing to run models like LLaMA, Mistral, etc., on personal or on-premise machines (useful for offline development, privacy, and cost)
 - llama.cpp – Optimized for GGUF format, CPU/GPU efficient.
 - Ollama – Uses ‘Modelfiles’ for model setup, easy to run and manage, runs server and client locally (CPU or GPU).
 - vLLM – High-throughput, low-latency engine for serving transformer models.
 - Hugging Face TGI (Text Generation Inference) – Standardized high-performance serving solution.
- **Commercial Servers (Proprietary APIs)** offering high-performance inference for large-scale LLMs:
 - Anthropic → Claude LLMs, OpenAI → GPT models,
 - Hugging Face Inference OpenRouter and Groq → inference for many LLMs
- **Cloud-based, “Serverless” Solutions** providing fully managed endpoints (plus often data storage)
 - Amazon Bedrock, Google Cloud AI, Microsoft Azure Databricks, Hugging Face Inference Endpoint